

Polygonal Objects Rendering (1)

Contents

- Basic renderer
- Rendering pipeline
- Local coordinates system
- World coordinates system
- View coordinates system
 - View transformation
 - View system specification in GUI
 - Back face culling
- Screen coordinate system
 - Perspective transformation
 - Orthogonal transformation
 - View volume

Rendering sequence

- **Basic renderer** - the traditional approach to render 3D objects.
- Polygonal renderer
 - Software and hardware
 - Process a polygon as a single entity -> fast and simple processing
 - Input: list of polygons
 - Output: color for each pixel on the screen
- The renderer incorporates a local reflection model (e.g. Phong model) into a Phong incremental shader.

Polygonal modeling

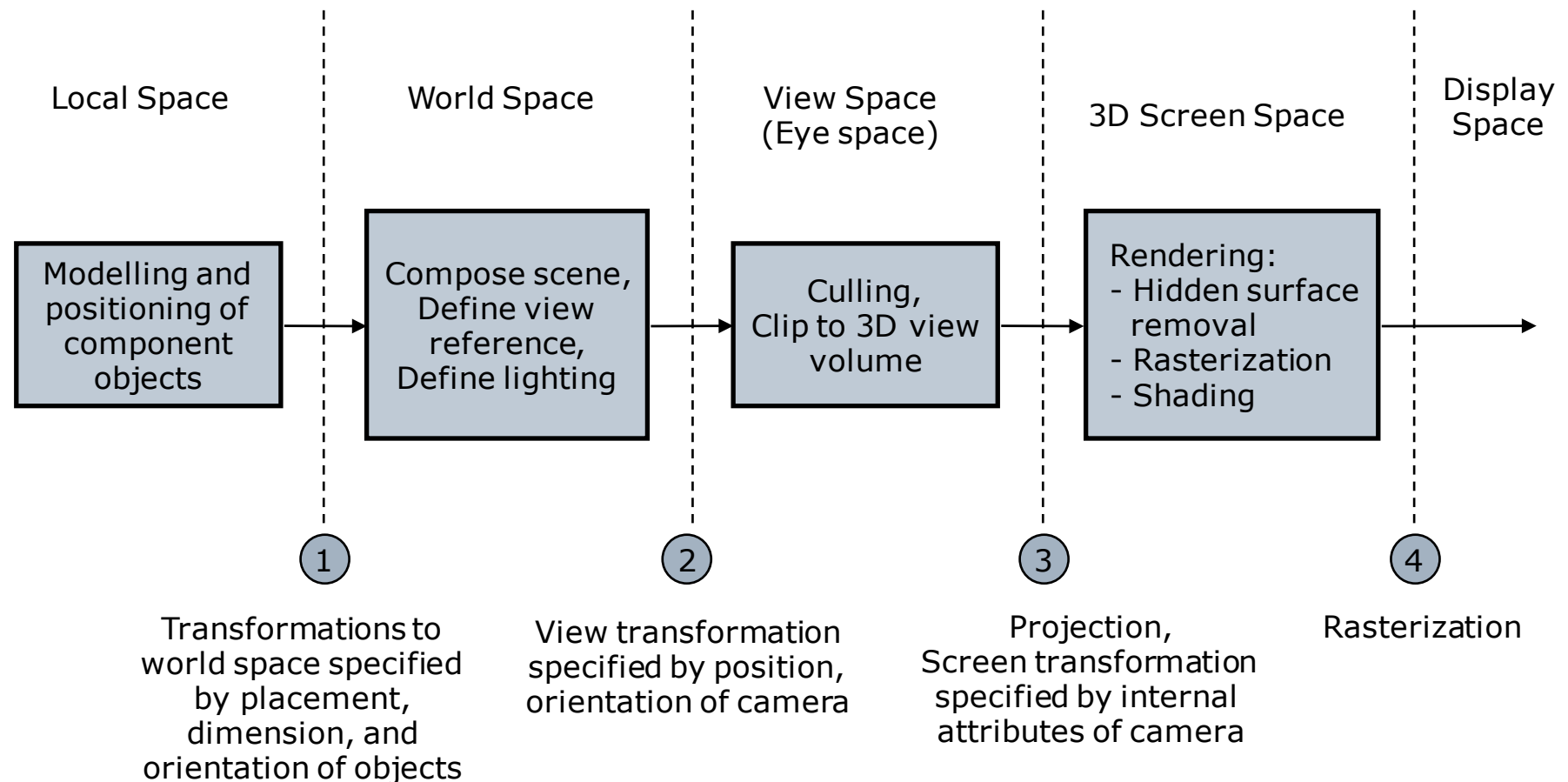
□ Advantages of the polygonal representation:

- Modelling objects using polygons is straightforward, simple but tedious.
- Geometric information is only stored at the polygon vertices.
- A polygonal based renderer can be used to render objects that are made up of a set of bicubic patches.

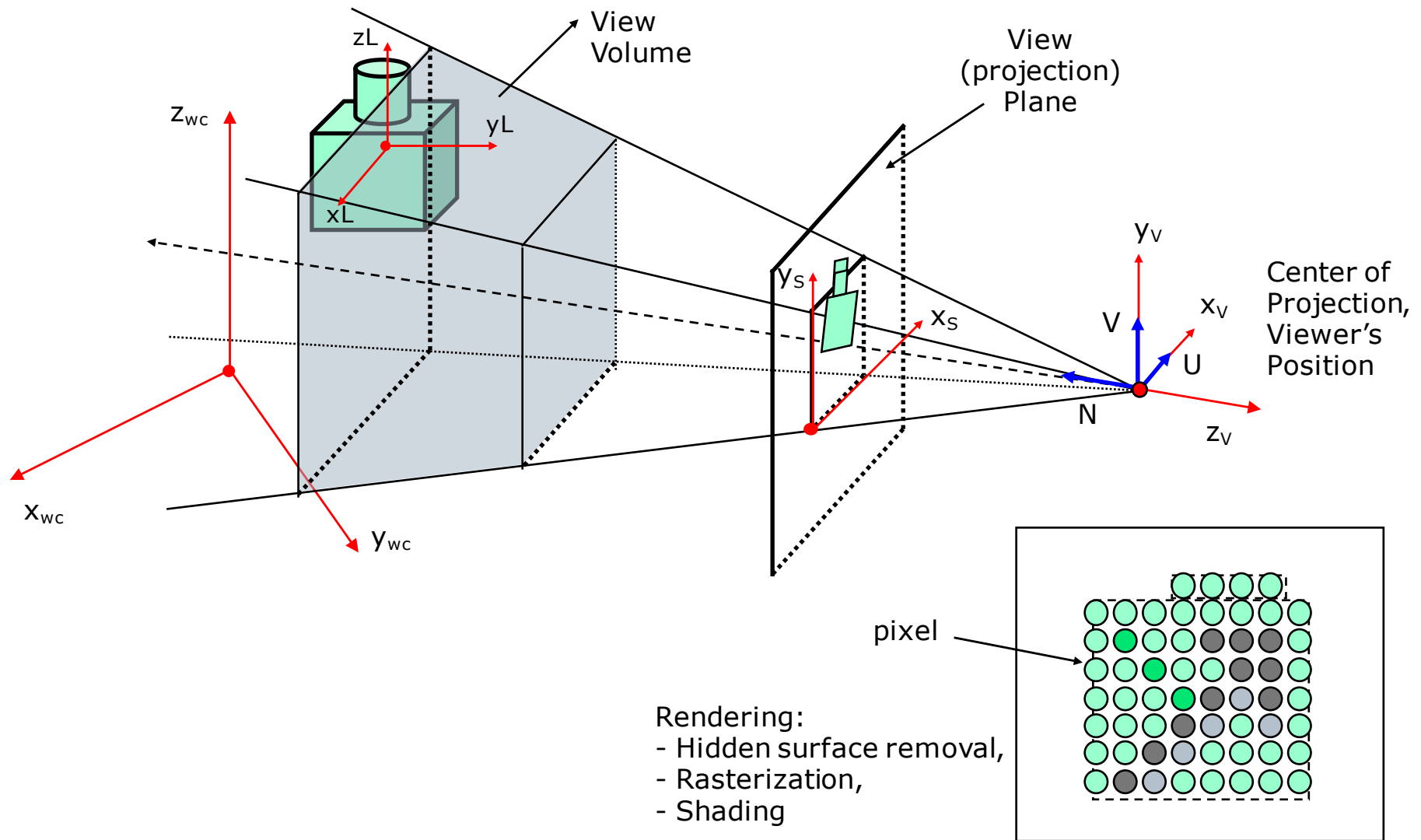
□ Disadvantages:

- It is difficult to map textures from a 2D domain onto the surface of an object
- Shadow algorithms that are based on polygonal objects generally have high coding complexity and produce hard-edge shadows.
- Polygons are expensive. A complex object that is to be represented to a high level of detail can generate a very large number of polygons (i.e. 400 000 polygons).

3D Objects rendering pipeline

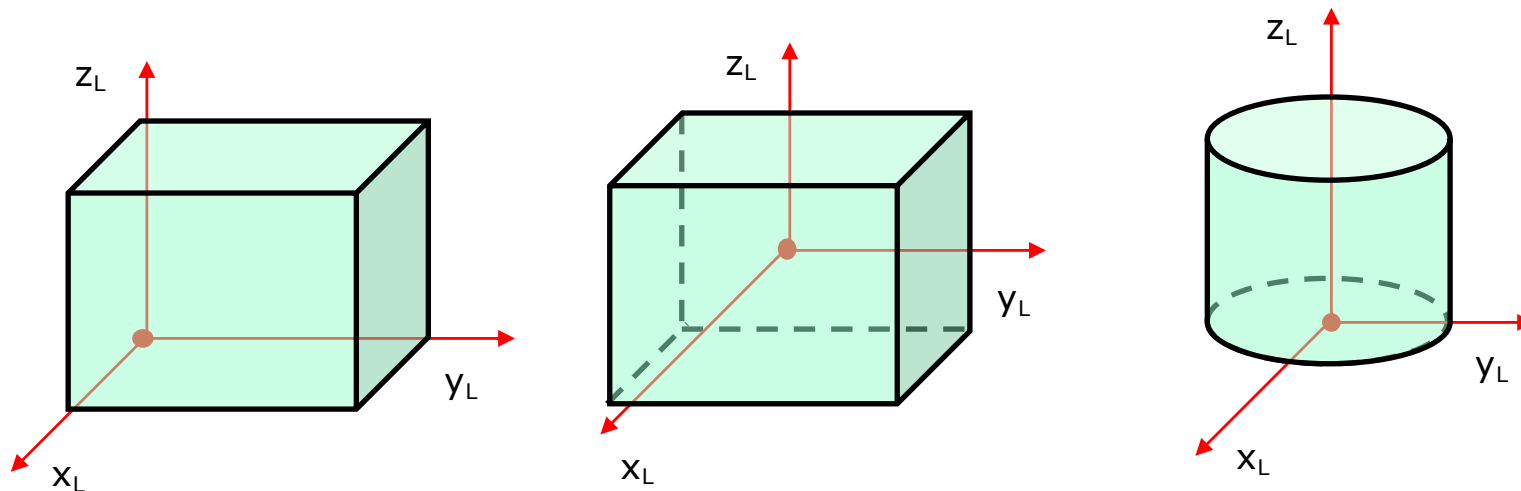


Conceptual model of 3D viewing process



Local coordinates system

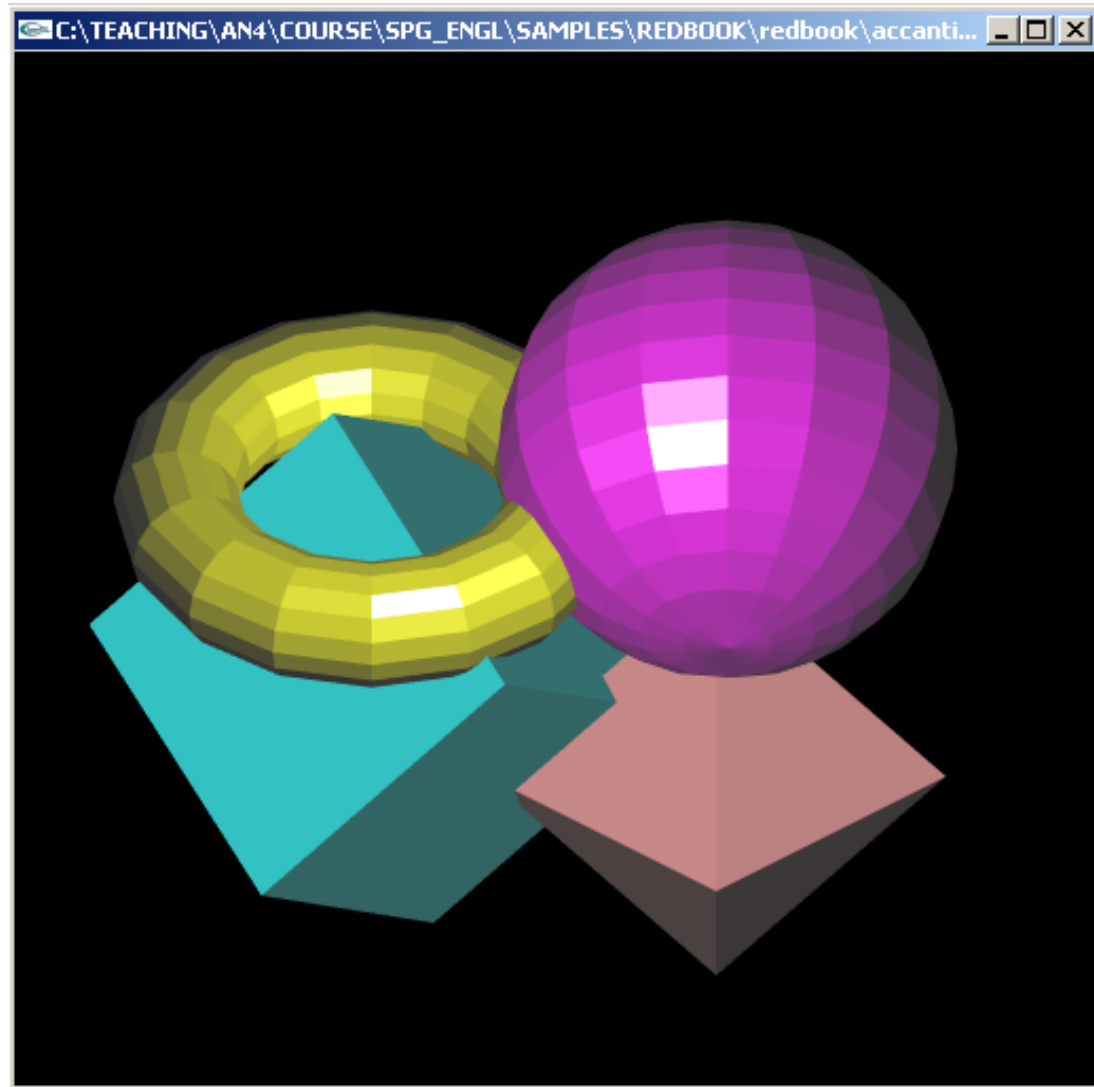
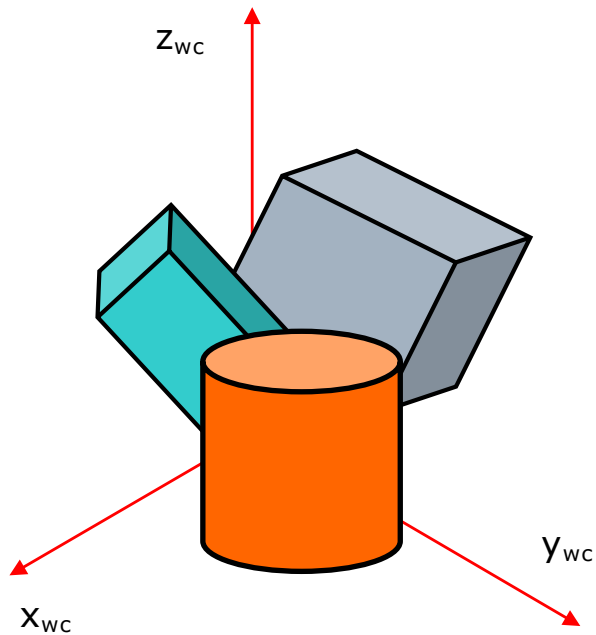
- The graphics model store the vertices of an object with respect to some point conveniently located in or near the object
- As well as storing the polygon vertices in a coordinate system that is local to the object, we would also store the polygon normals and the vertex normals.
- Example: in PHIGS the local coordinates system is called the **modelling coordinates system**



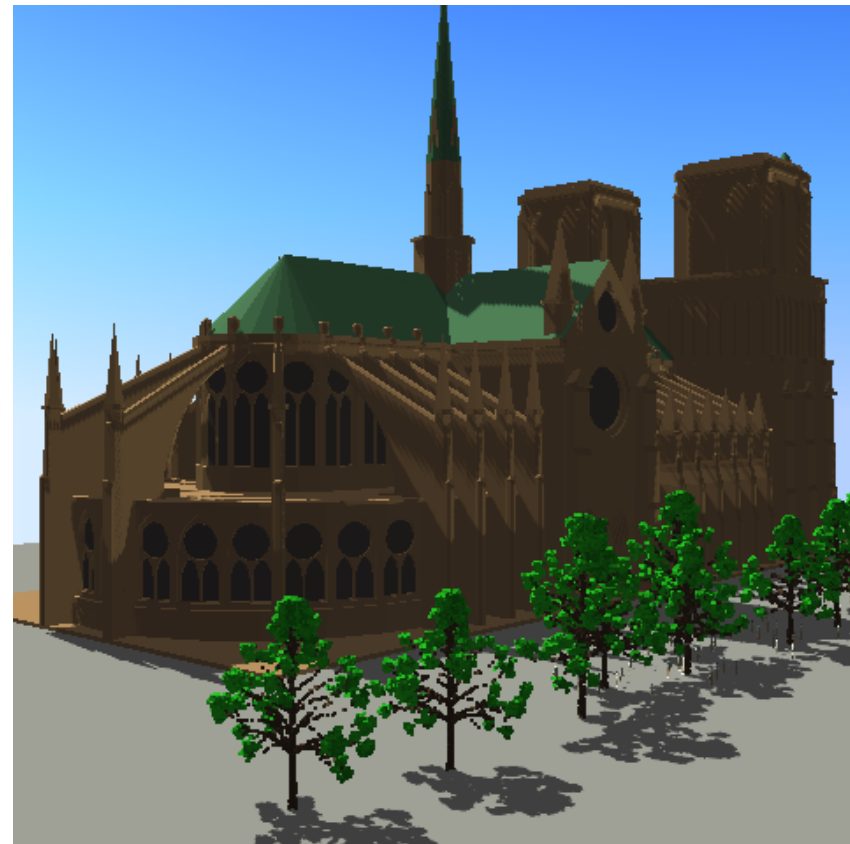
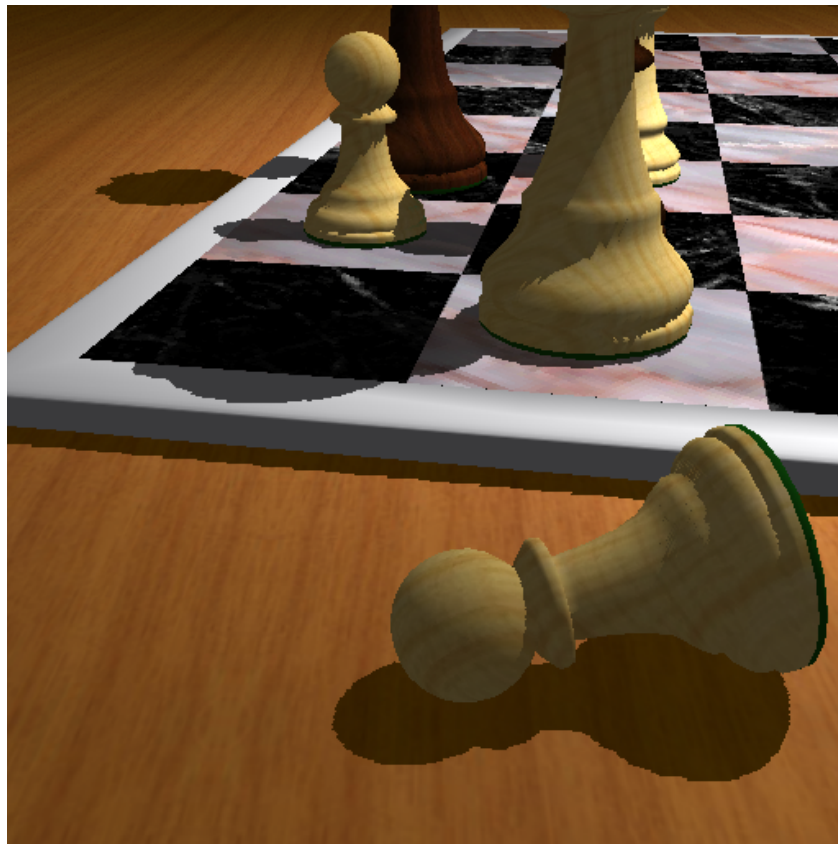
World coordinates system

- ❑ The objects are placed into a scene of objects
- ❑ The global coordinate system of the scene is known as the **world coordinate system**. The act placing an object in a scene defines the **transformation** required to take the object from local space to world space
- ❑ If this object is being animated then the system provides a time-varying transformation that takes the object into world space on a frame-by-frame basis
- ❑ **Light sources** are specified for the scene that is lit in world space.
- ❑ **Surface attributes** (i.e. texture, colour, etc) are specified in world space as well

World coordinates system



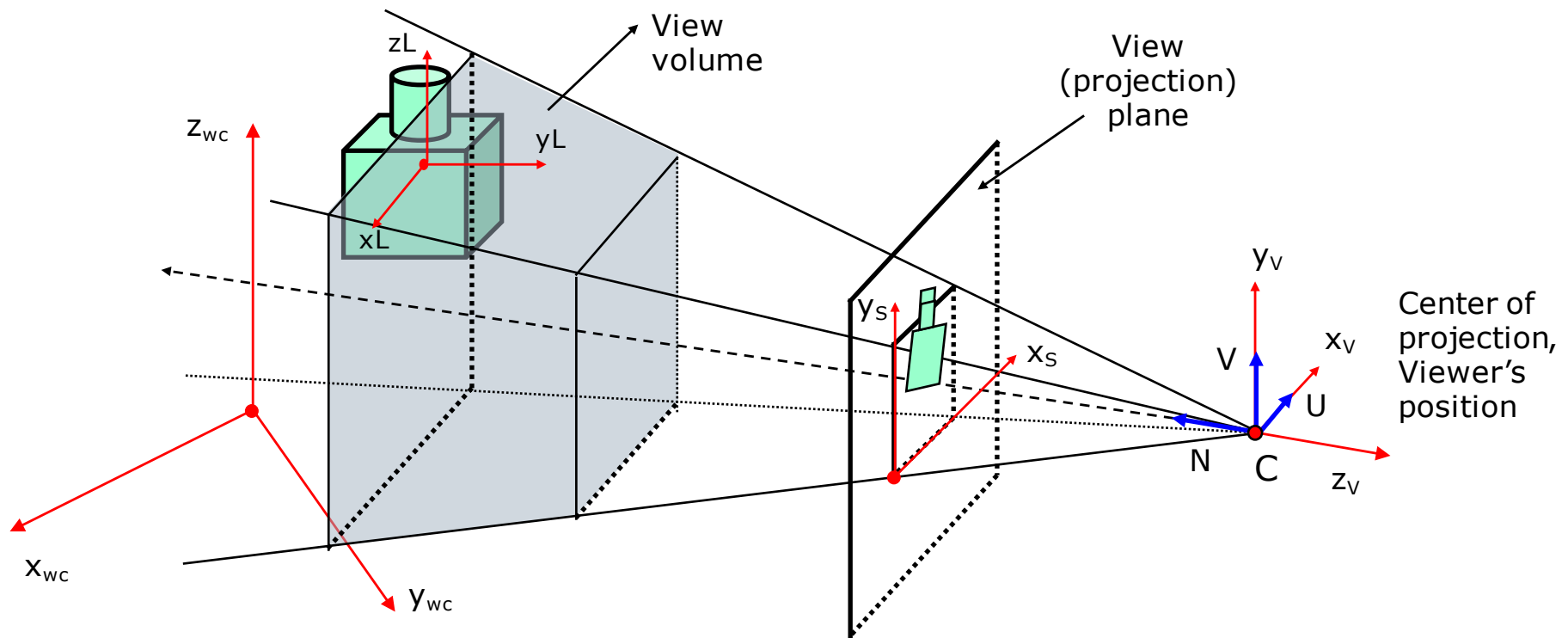
World coordinates system



View coordinates system

- Eye, view or camera coordinates system
- It is the space used to establish **viewing parameters** and a **view volume**. A **virtual camera** is often used as a conceptual aid in computer graphics
- The camera can be positioned anywhere in world space and point in any direction. The analogy of the film plane in computer graphics is the **view plane** - the plane in which the scene is projected
- Example: in PHIGS the viewing system is called "view reference coordinate system", or the **(U,V,N) system**

Conceptual model of 3D viewing process



View system definition

□ 3 vectors and a position:

1. **Camera position, C.** This is also the centre of projection.
2. **Viewing direction vector, N.** Along z axis, a vector normal to the view plane. In OpenGL the N vector gives the $-z$ axis.
3. **"Up" vector, V** - orients the camera about the view direction and establish with N the orientation of the view plane window within the view plane. The V vector gives the $+y$ axis.
4. **Optional vector, U** - denotes the direction of increasing x in the eye coordinate system. This establish a right ($Z = X \times Y$) or left ($-Z = X \times Y$, or $Z = Y \times X$) handed coordinate system.

In OpenGL:

- The view vector system is left handed $U = N \times V$, $N = V \times U$
- N vector gives the $-z$ direction, and is oriented toward the scene

Transformation from world to view system

- The transformation is given by the following equation:

$$\begin{bmatrix} x_v & y_v & z_v & 1 \end{bmatrix} = \begin{bmatrix} x_w & y_w & z_w & 1 \end{bmatrix} * H$$

where $H = T * R$, T is the translation to the C centred system, and R is a complex rotation which expresses the rotation into the view system

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_c & -y_c & -z_c & 1 \end{bmatrix}$$

R maps U , V and N to unit and orthogonal vectors:

$$U \rightarrow (1, 0, 0)$$

$$V \rightarrow (0, 1, 0)$$

$$N \rightarrow (0, 0, -1)$$

Transformation from world to view system

or U, V, and N in the matrix form are transformed to unit vector:

$$\begin{bmatrix} U_x & U_y & U_z & 1 \end{bmatrix} * R = \begin{bmatrix} 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} V_x & V_y & V_z & 1 \end{bmatrix} * R = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} N_x & N_y & N_z & 1 \end{bmatrix} * R = \begin{bmatrix} 0 & 0 & -1 & 1 \end{bmatrix}$$

U, V and N are unit and orthogonal vectors:

$$U_x^2 + U_y^2 + U_z^2 = 1$$

$$U_x V_x + U_y V_y + U_z V_z = 0$$

$$V_x^2 + V_y^2 + V_z^2 = 1$$

$$N_x V_x + N_y V_y + N_z V_z = 0$$

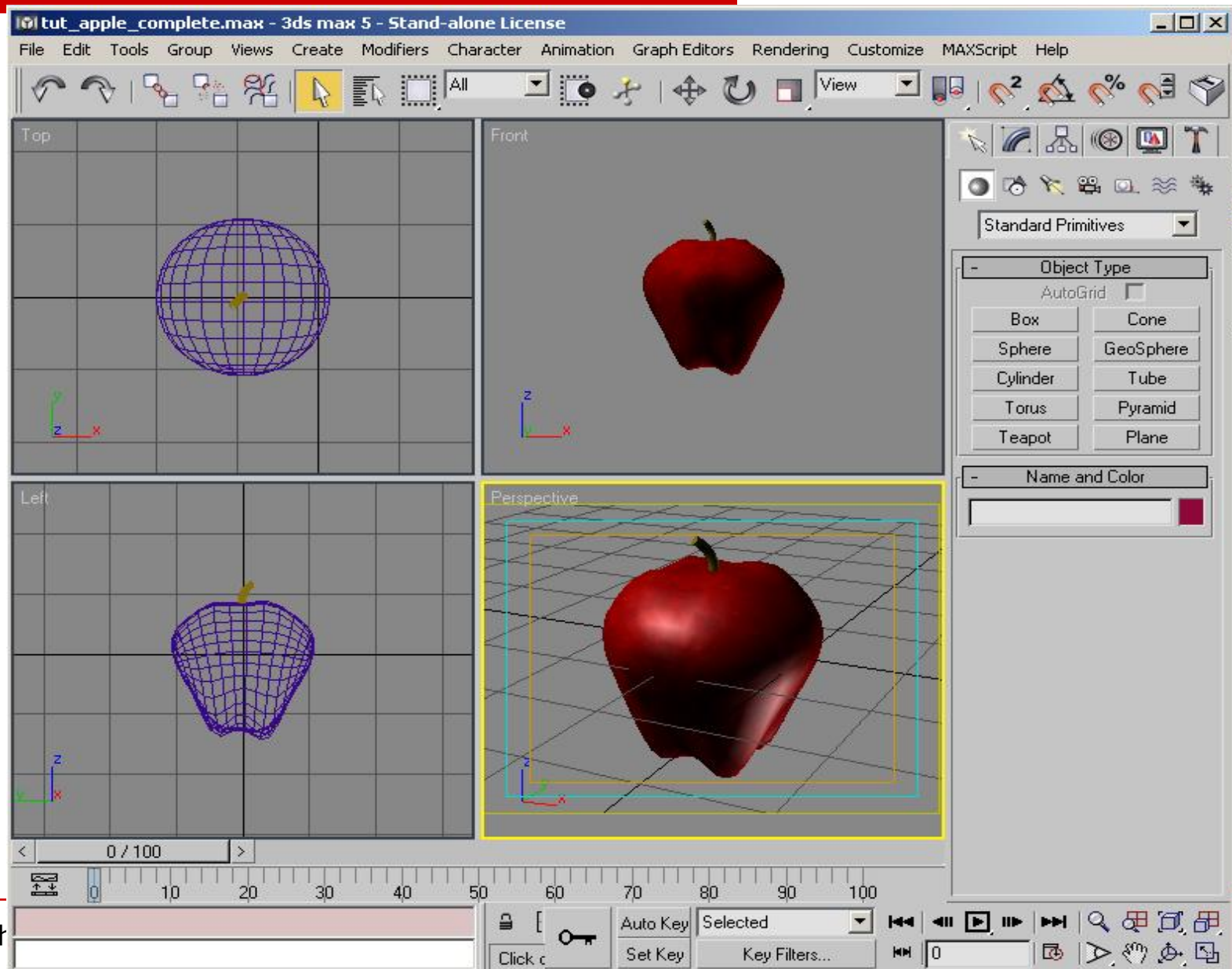
$$N_x^2 + N_y^2 + N_z^2 = 1$$

$$U_x N_x + U_y N_y + U_z N_z = 0$$

Therefore:

$$R = \begin{bmatrix} U_x & V_x & -N_x & 0 \\ U_y & V_y & -N_y & 0 \\ U_z & V_z & -N_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

View system specification in GUI



Graph

View system specification in GUI

- ❑ Objective: to control the viewer's position and viewing parameters in 3D applications.
- ❑ Viewing parameters C, U, V and N
- ❑ Position $C(x_C, y_C, z_C)$

x, y (2D screen space) $\rightarrow C, U, V, N$ (3D space)

- ❑ Vector N definition:

$$N_x = \sin \beta \cos \alpha$$

$$N_y = \sin \beta \sin \alpha$$

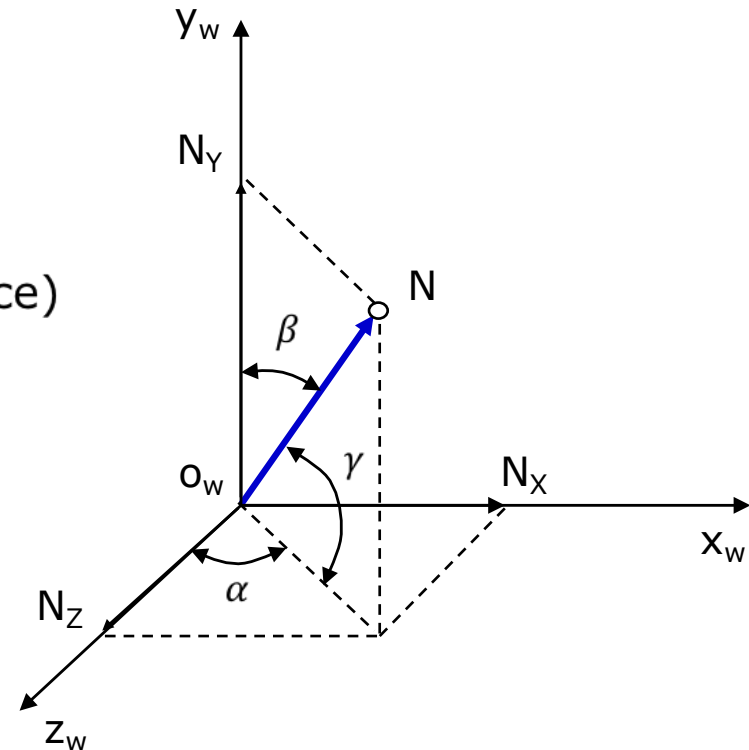
$$N_z = \cos \beta$$

- ❑ or as function of γ

$$N_x = \cos \gamma \cos \alpha$$

$$N_y = \cos \gamma \sin \alpha$$

$$N_z = \sin \gamma$$



View system specification in GUI

□ Vector V definition:

1. V must be perpendicular to N. We have to determine the intersection of the (N, V') plane and the viewing plane. The planes are perpendicular.
2. Draw the vector $V'' = V' \times N$. V'' is in the projection plane and perpendicular on the (N, V') plane.
3. Finally V is the normalized cross product of N and V'', $V^* = N \times V''$.

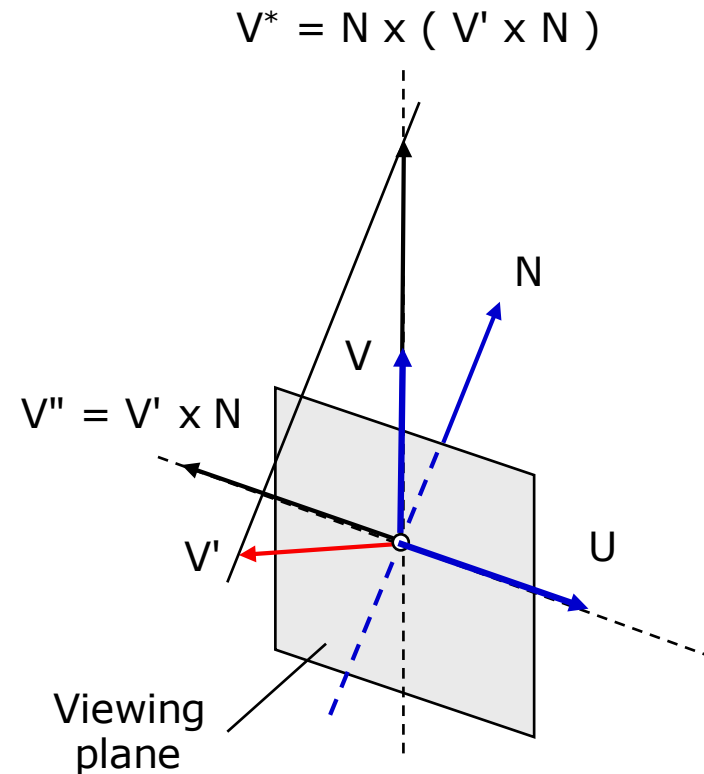
□ Conclusion:

V is the normalized vector

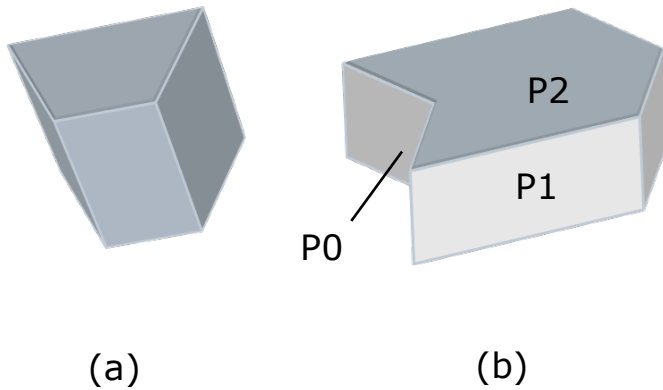
$$V^* = N \times (V' \times N)$$

□ The vector U is the cross product

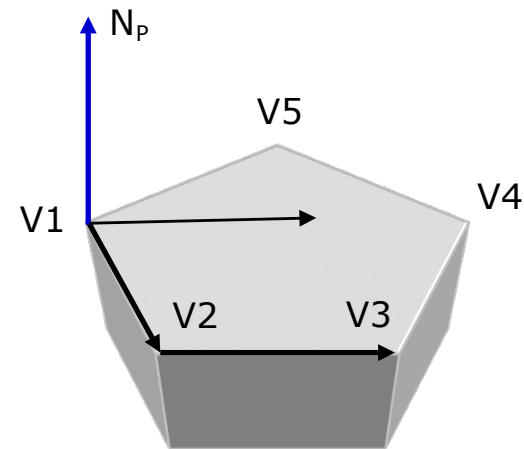
$$U = N \times V.$$



Back face culling



(a) Convex, and (b) concave objects



The computation of the polygon normal vector

Back face culling – visibility test

$$\mathbf{N_p} \cdot \mathbf{L} = |\mathbf{N_p}| \cdot |\mathbf{L}| \cos \varphi$$

There are 3 visibility cases:

(a) Visible polygon face

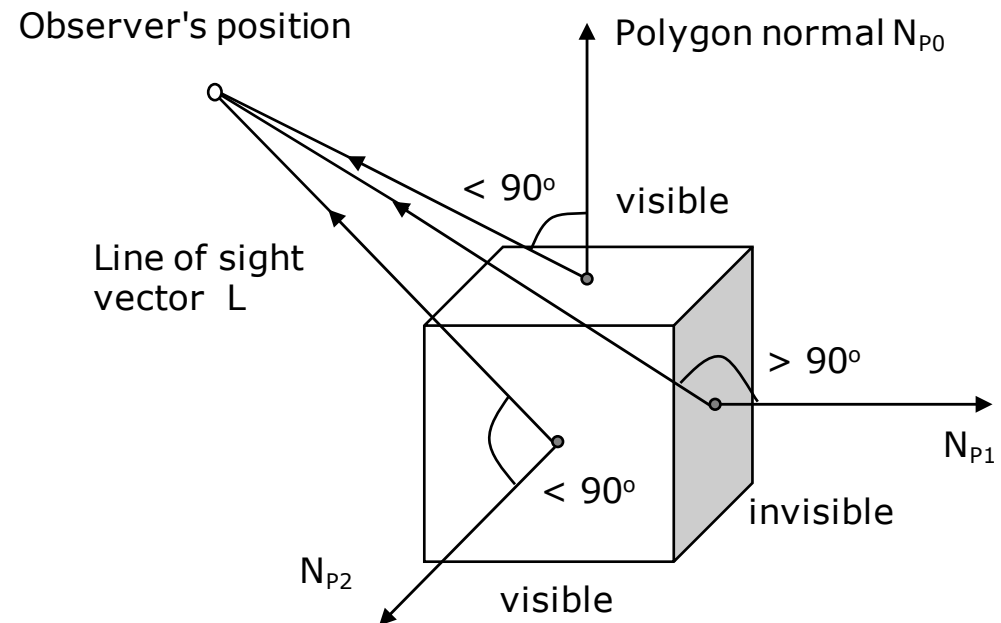
$$\varphi < 90^\circ, \quad \cos \varphi > 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} > 0$$

(b) Observer's line

$$\varphi = 90^\circ, \quad \cos \varphi = 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} = 0$$

(c) Invisible polygon face

$$\varphi > 90^\circ, \quad \cos \varphi < 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} < 0$$

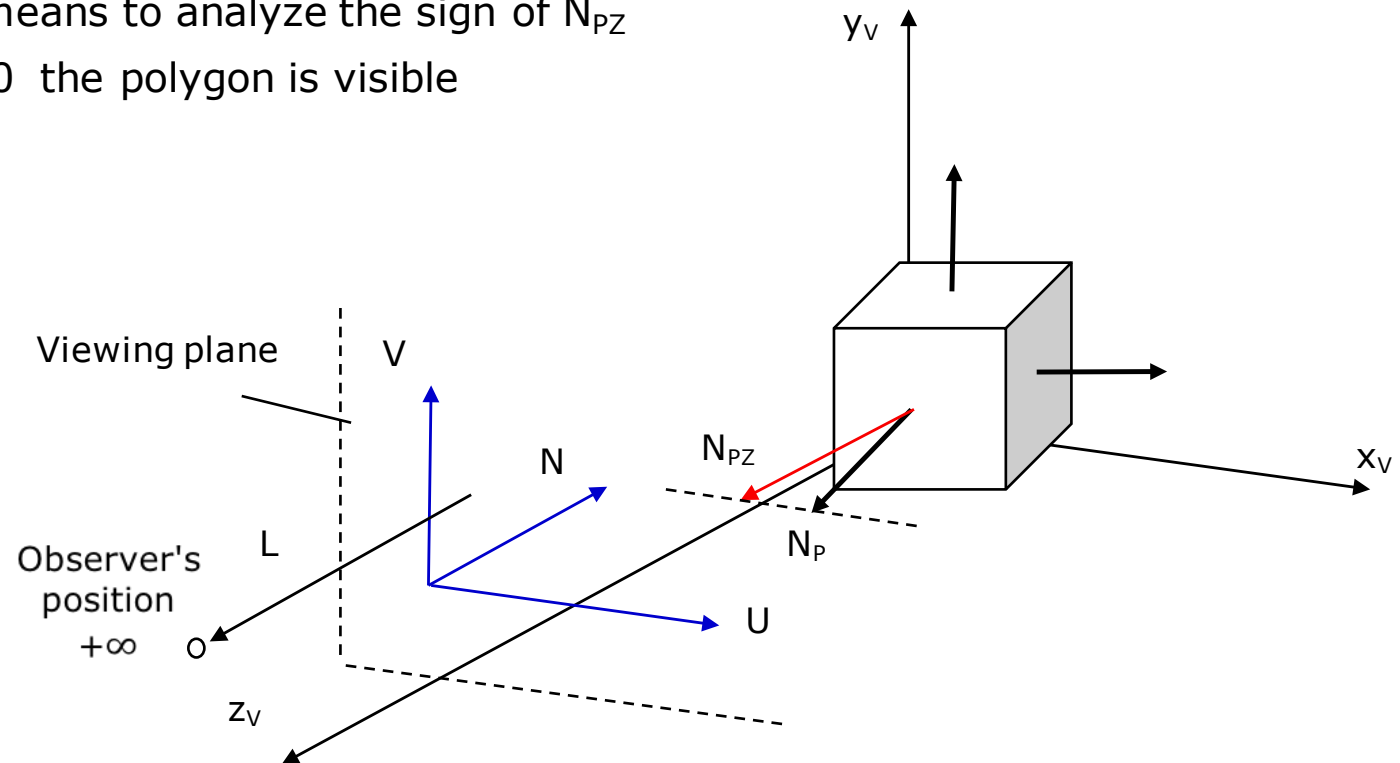


Computings:

1. Dot product: $\mathbf{N_p} \cdot \mathbf{L} = N_{pX} L_X + N_{pY} L_Y + N_{pZ} L_Z$
2. Line of sight vector L
3. Polygon normal vector: $\mathbf{N_p} = [\mathbf{V_1} \ \mathbf{V_2}] \times [\mathbf{V_2} \ \mathbf{V_3}]$

Accelerating the visibility test

- Perspective (view space) and orthogonal projection (screen space)
- Observer's position at infinity on the z_v axis
- The line of sight L is the opposite vector of N
 $N = [0 \ 0 \ -1]$, $V = [0 \ 1 \ 0]$, $U = [1 \ 0 \ 0]$
- Visibility test means to analyze the sign of N_{pz}
If $N_{pz} > 0$ the polygon is visible

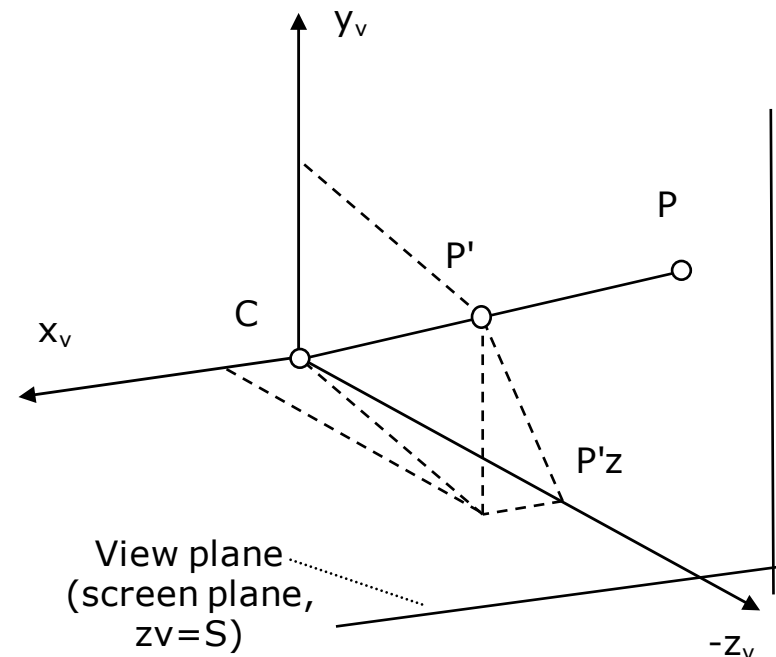
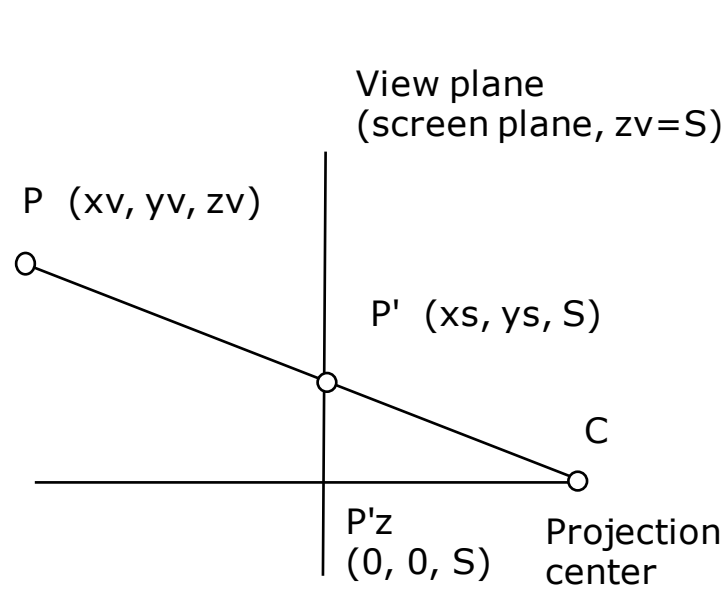


Screen space

- ❑ Screen space is a 3D space
- ❑ The value of x_s and y_s coordinates in the screen space are equal to the coordinate values in the view plane and not normalized screen coordinate values
- ❑ The z coordinate is not constant. It keeps the depth information to provide depth based computation in the screen space
- ❑ For simplicity, we use the expression of screen coordinate instead of coordinate in the screen space

Perspective projection to screen plane

- Screen coordinates: $x_s = S \cdot \frac{x_v}{z_v}$
- $y_s = S \cdot \frac{y_v}{z_v}$



Perspective transformation

To express the non linear transformation

$$xs = S \cdot \frac{xv}{zv}$$
$$ys = S \cdot \frac{yv}{zv}$$

as a 4x4 matrix consider the [homogeneous coordinates](#) and the transformation in two parts: a linear part (1°), followed by a non-linear part (2°):

1° [Linear transformation](#): $X=xv$, $Y=yv$, $Z=zv$, $w=zv/S$

The perspective transformation can be written as: $\begin{bmatrix} X & Y & Z & w \end{bmatrix} = \begin{bmatrix} xv & yv & zv & 1 \end{bmatrix} T_{\text{pers}}$

$$T_{\text{pers}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/S \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

2° [Non-linear transformation](#): perspective division by w .

The final screen coordinates are: $xs=X/w$, $ys=Y/w$, $zs=Z/w$

Orthogonal transformation

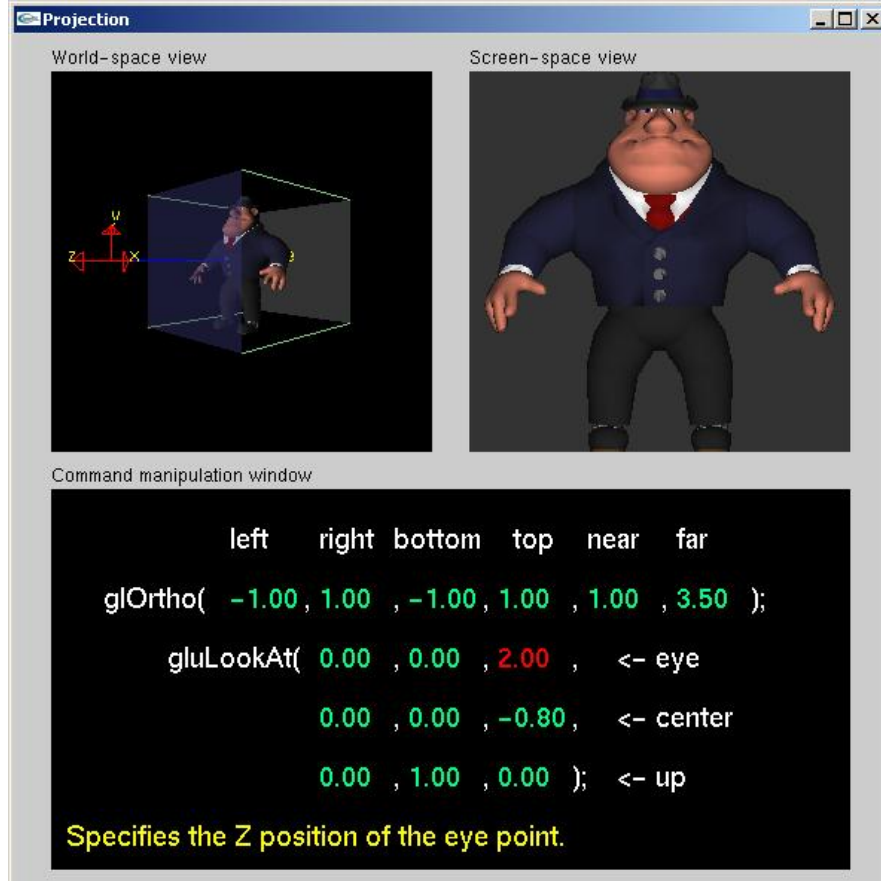
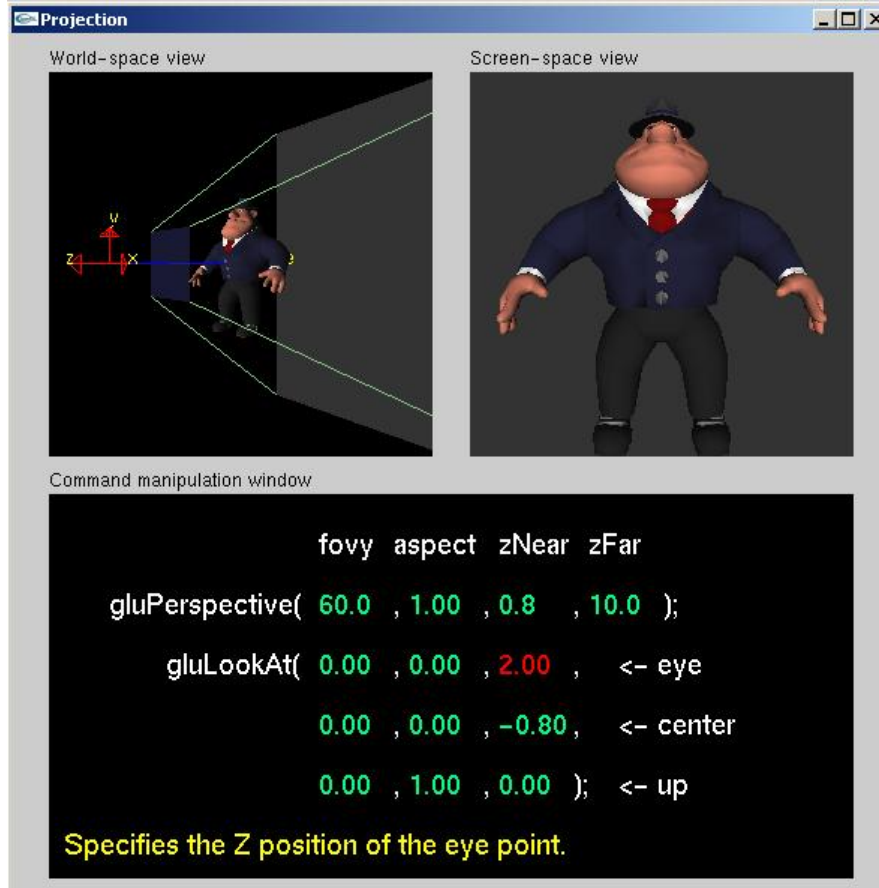
- Parallel projection
- The view plane is normal to the direction of projection

$$x_s = x_v, \quad y_s = y_v, \quad z_s = 0$$

and the orthogonal transformation is:

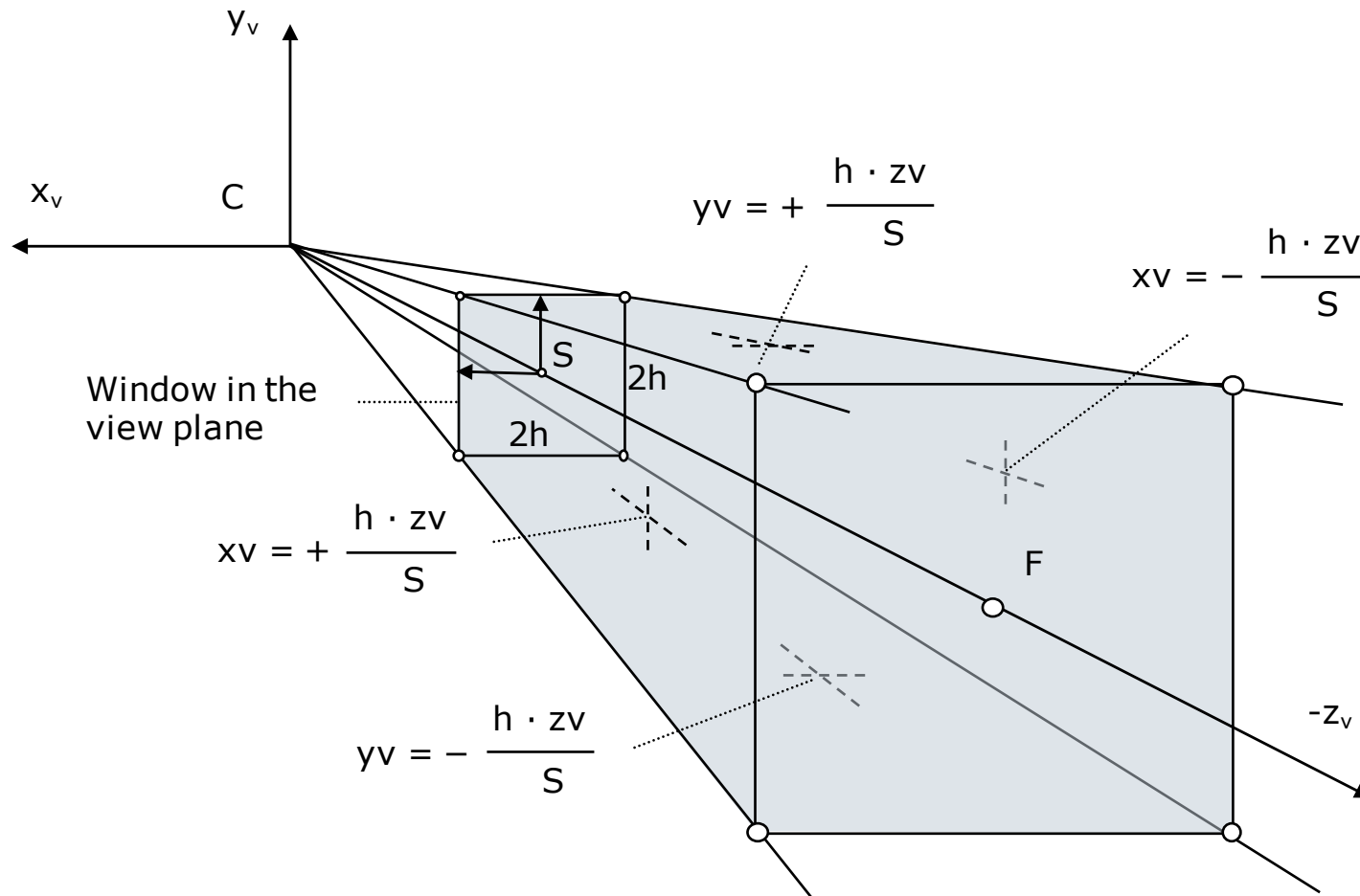
$$T_{\text{orth}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Perspective and orthogonal transformation



View volume

- Screen space is defined inside the view volume (viewing frustum)



View volume

- View volume definition: 6 planes

$$z_v = S$$

$$z_v = F$$

$$x_v = \mp \frac{h \cdot z_v}{S}$$

$$y_v = \mp \frac{h \cdot z_v}{S}$$

- $2h$ is the view plane window dimension
- S is the view plane distance, the virtual screen plane distance, and the near plane distance
- F is the far plane distance
- S and F are application based parameters

Questions and problems

1. What are the main advantages and disadvantages of the polygonal models?
2. What phases and coordinate system spaces, along the graphics pipeline, are able to support the following operation:
 1. Shadow computation;
 2. 3D clipping;
 3. Texture mapping;
 4. 2D translation;
 5. 3D rotation?
3. Exemplify and explain the transformations from world to view system.
4. Explain why is difficult to specify in GUI the viewing direction for the viewer.

Questions and problems

5. Explain why in orthogonal projection the visibility test for a polygonal phase is reduced to the analyzing of N_z (the z component of the normal vector)?